Nick Darby and Sarina Talerico
Professor Han
10/11/2026
DSCI 255

Written Report for Water Image Sample Analysis

Abstract

Harmful cyanobacterial blooms threaten drinking water and local water bodies, but monitoring them depends on looking at samples under a microscope by hand, which is slow. In collaboration with the City of Worcester and WCMC, and with help from Prof. Nathan Ahlgren, our project builds the tools needed to use machine learning on FlowCam images of local water samples. A FlowCam is a microscope that takes pictures of particles in flowing water. It produced hundreds of thousands of small images, saved in folders with inconsistent names, which made them hard to search, compare, or use to train a model.

First, we wrote a Python script that reads each folder name and pulls out details like the site, date, magnification, and dilution. It also fixes inconsistent site names, picks the processed version of each folder, and checks that every image opens. The final catalog has 511,765 images from 613 folders, 45 sites, and 36 sampling dates, and none were unreadable. Next, we built a website using Supabase for the database and logins and Cloudflare for hosting and image storage. Anyone can browse, filter, and download images, and approved users can label them. The labeling page supports selecting many images at once, keyboard shortcuts, undo, progress tracking, and CSV export, and the database records who labeled each image and when. We also added dark mode and a demo mode that works without an account. Teammates also built a FlowCam file decoder and tested machine learning models.

Together, these pieces turn raw FlowCam output into an organized, searchable dataset that people can label. Our next steps are to finalize the label categories with experts, launch the labeling tool, and use the labeled images to train and test models that detect cyanobacteria, making water monitoring faster and more consistent.

Introduction and Background

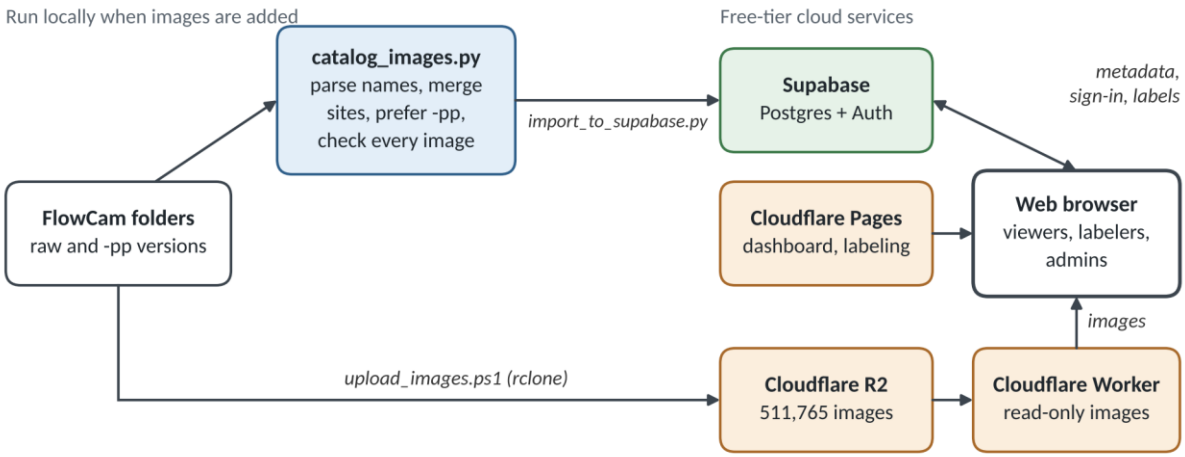
Some cyanobacteria produce toxins that endanger people and pets when their populations bloom, and blooms are becoming more frequent as nutrient pollution and warming favor cyanobacterial growth (Paerl & Huisman, 2009). The Worcester Cyanobacteria Monitoring Collaborative (WCMC) samples about 20 Worcester ponds roughly every two weeks, around 240 samples per year. Microscope identification remains the gold standard but is slow and requires experts.

WCMC also runs samples through a FlowCam, which photographs each particle as water flows past its camera and produces hundreds of images per sample in minutes. This speeds up imaging but not identification. Machine learning could automate identification, and a former student of Prof. Ahlgren trained an initial classifier on a small hand-sorted subset (Francis, 2026), but scaling up requires the whole archive to be organized and labeled. When our project began, the archive was a set of hand-named folders with inconsistent site spellings, dates, and

settings, and many samples existed twice, as a raw export and as a post-processed ("-pp") version.

Our goal was to turn this archive into an organized, searchable dataset that experts can label. This mirrors the first two stages of the open-source FlowCam pipeline of Symiakaki et al. (2026): preprocess instrument output into one table, then annotate it with a labeling tool, before optional machine learning classification.

Methodology



System architecture. Local scripts catalog the FlowCam folders, load metadata into Supabase, and upload images to Cloudflare R2. Visitors load the site from Cloudflare Pages, read metadata and save labels through Supabase, and view images through a read-only Worker.

Cataloging script

Prof. Ahlgren provided the archive as a directory of sample-event folders of PNG images, one folder per FlowCam run. Each folder name encodes the sample, for example Coes Reservoir_9.16.2023_10x_TM 0.5 dilution-pp. Our Python script, catalog_images.py, reads every folder without modifying any files:

- Parsing. Regular expressions extract the site, date, sample code, dilution, and -pp status. Sample codes are grouped into autoimage (AI) and trigger (TR, TM) capture modes, and water-body type (pond, lake, reservoir, park, shore) is read from the site name.
- Magnification. Despite the abstract's wording, magnification is not read from folder names, because many omit it. Prof. Ahlgren confirmed these were imaged at the standard 10x, so every folder is recorded as 10x.
- Site normalization. Glued words are split (ElmPark becomes Elm Park) and spelling variants merged, reducing 116 raw site spellings to 45 sites, checked against Prof. Ahlgren's folder spreadsheet.
- Deduplication. When a raw folder has a -pp counterpart (same site, date, and sample code), only the -pp folder, the city's standard output, is kept. A test-import folder that duplicated a real sample is excluded.

- Validation. Every image is opened with the Pillow library to record its size and format and confirm it is readable.

The outputs are image_manifest.csv, one row per image with an ID derived from its path (so labels survive reruns) and an empty label column, and folder_summary.csv, one row per folder.

Hosting and database

Metadata and accounts live in Supabase (Figure 1), a hosted PostgreSQL database with Google or email sign-in. Images are stored in Cloudflare R2 and served read-only by a Cloudflare Worker, and the plain HTML, CSS, and JavaScript site is hosted on Cloudflare Pages, rebuilt on every push to GitHub (DS-Studio-F26, 2026). import_to_supabase.py loads the manifest in one transaction that never touches labels and refuses to commit if counts do not match, and upload_images.ps1 copies only new images to R2. Permissions are enforced in the database with Row-Level Security: anyone can read, new accounts start as viewers, admins assign roles, and labelers can change only an image's label.

Labeling tool

The labeling page lists the least-labeled folders first. Labelers select many image tiles at once and apply a label with one key press, or use a zoomable viewer for one-by-one work. Saves happen in the background, roll back if they fail, and can be undone, and a database trigger records who labeled each image and when, so attribution cannot be faked from the browser.

Pathways considered but not used

Our Plan B for hosting ($7 per month) paired a Render backend server and Render Postgres database with the same Cloudflare storage. We chose Plan A ($0 per month) because it has no server to maintain and lets the database enforce permissions directly. Our first dashboard ran only on one laptop, reading per-folder JSON files from prepare_image_data.py; the hosted site replaced it because it could not be shared or support multiple labelers.

Results

Table 1. Final image catalog by capture mode, water-body type, and sampling year. No images were unreadable.

Category	Subset	Folders	Images
All	All cataloged images	613	511,765
Capture mode	Trigger (TR, TM codes)	493	337,971
	Autoimage (AI codes)	120	173,794
Water-body type	Pond	356	277,964
	Lake	132	90,311
	Reservoir	54	32,621
	Park, shore, or not identifiable	71	110,869
Sampling year	2021 (5 sampling dates)	55	71,884
	2022 (15 sampling dates)	225	365,181
	2023 (15 sampling dates)	316	70,002
	2024 (1 sampling date)	17	4,698

The final catalog contains 511,765 images from 613 folders, covering 45 sites and 36 sampling dates from June 2021 to June 2024, and every image opened successfully (Table 1). Deduplication dropped 33 raw folders in favor of their -pp versions; 573 of the 613 remaining folders are post-processed. The data are uneven: ponds hold 54% of images, trigger mode 66%, and three sites (Elm Pond, Burncoat Pond, and Indian Lake) 37%. Although 2022 has fewer folders than 2023, it has over five times as many images.

The website is live at water-sample-dashboard.pages.dev, where anyone can filter folders by seven attributes (site, water-body type, date, sample code, capture mode, dilution, processing status) and view or download images. The images use 3.65 GiB of R2's 10 GB free tier and the database about 168 MB of Supabase's 500 MB, so the system costs nothing to run. The labeling tool is deployed and tested in demo mode; no real labels exist yet because the categories are still being decided.

Discussion

WCMC's archive is now a dataset rather than a pile of folders. Tasks that meant manual folder hunting, such as finding every trigger-mode image from one pond across a season, are now a website filter or a single query, and stable IDs keep labels attached as new seasons are added. Table 1 also shows what a future classifier will face: a model trained on random images would mostly learn 2022, trigger mode, and a few large ponds. Labeling sets should be balanced across sites, dates, and capture modes, and test sets should hold out whole samples or dates, since images from one sample look alike and random splits overstate accuracy.

Limitations

- Metadata is only as good as folder names: dilution is missing for 225 of 613 folders and water-body type for 51, some dilution spellings ("1dilution" vs. "1 dilution") still appear as separate filter options, magnification is assumed rather than read, and site merging is rule-based, so a wrong merge would go unnoticed.
- The label categories (cyanobacteria, diatom, other organism, debris, blank, unsure) are placeholders, and no real labels exist yet.
- Free tiers impose limits: Supabase pauses after about a week without activity, and its email sender allows only a few sign-up and password-reset emails per hour.

Future Work

1. Finalize the label categories with Prof. Ahlgren and WCMC experts.
2. Launch labeling, starting with a set balanced across sites, dates, and capture modes.
3. Train and test classifiers on the labeled images, holding out whole samples or dates, and store predictions separately from human labels so experts validate rather than relabel (Symiakaki et al., 2026).

Conclusion

We turned WCMC's inconsistently organized FlowCam archive into a validated catalog of 511,765 images across 45 sites and 36 sampling dates, and built a free, secure website where anyone can explore the data and approved users can label it. The project does not yet detect

cyanobacteria, but it provides the organized dataset and labeling tool a reliable classifier requires.

References

Cloudflare. (2026). Cloudflare R2, Workers, and Pages [Computer software]. https://developers.cloudflare.com/

DS-Studio-F26. (2026). water-sample-image-organization-and-analysis [Computer software]. GitHub. https://github.com/DS-Studio-F26/water-sample-image-organization-and-analysis

Francis, K. (2026). cyanobacteria-ovr-classifier [Computer software]. GitHub. https://github.com/KaviFrancis/cyanobacteria-ovr-classifier

Paerl, H. W., & Huisman, J. (2009). Climate change: A catalyst for global expansion of harmful cyanobacterial blooms. Environmental Microbiology Reports, 1(1), 27–37. https://doi.org/10.1111/j.1758-2229.2008.00004.x

Supabase. (2026). Supabase [Computer software]. https://supabase.com/

Symiakaki, K., Walles, T. J. W., Park, C., Yalçın, G., Siangsano, K., Berger, S. A., & Nejstgaard, J. C. (2026). Pipeline for FlowCam data processing with modular open-source software and optional machine learning classification. PeerJ, 14, e20754. https://doi.org/10.7717/peerj.20754